



CYBERSECURITY FOR SOFTWARE DEVELOPMENT

Unit Code - B9CY104

Lecture – Tejas Bhat

Prepared by

ANSHIO RENIN MICHEAL ANTONY XAVIER SOOSAMMAL - 20036753

Submitted Date: 04/03/2025

TABLE OF CONTENTS

SI.NO	CONTENT	PAGE NO
1.	MANAGEMENT OVERVIEW	1
2.	SUMMARY OF KEY FINDINGS	1
3.	TOOLS AND SETUP	1
	3.1 SONARQUBE	1
	3.1.1 Installation Steps	2
	3.1.2 Start SonarQube	4
	3.1.3 Access SonarQube Web UI	4
	3.1.4 Install SonarScannert	4
	3.2 SEMGREP	5
	3.2.1 Installation Steps	5
	3.2.2 Basic Scanning	5
4.	APPLICATION OVERVIEW AND ANALYSIS	5
	4.1 MOIN MASTER	5
	4.2 HACKAZON	6
5.	REPORTED ISSUES	6
	5.1 SONARQUBE FINDINGS – MOIN MASTER	7
	5.2 SEMGREP FINDINGS – MOIN MASTER	8
	5.3 SONARQUBE FINDINGS – HACKAZON	8
	5.4 SEMGREP FINDINGS – HACKAZON	9
6.	COMPARATIVE ANALYSIS	10
	6.1 SONARQUBES STRENGTH	10
	6.2 SEMGREP STRENGTHS	11
	6.3 OVERLAP IN FINDINGS	12
	6.3.1 Similar Findings	12
	6.3.2 Unique Findings by SonarQube	12
	6.3.3 Unique Findings by Semgrep	12
	6.4 FALSE POSITIVE ANALYSIS	13
	6.5 TOOL INTEGRATION AND USABILITY	13
7.	CONCLUSION	13
8.	REFERENCE	14

9.	APPENDIX	14
	9.1 APPENDIX 1 - SONARQUBE	14
	9.2 APPENDIX 2 – MOIN MASTER WEB APPLICATION SONARQUBE	15
	9.3 APPENDIX 3 – HACKAZON MASTER WEB APPLICATION SONARQUBE	17
	9.4 APPENDIX 4 – MOIN MASTER SEMGREP SCAN	20
	9.5 APPENDIX 5 – HACKAZON SEMGREP SCAN	21

FIGURE OF CONTENTS:

FIGURE NO:	CONTENT	PAGE NO:
9.1.1	SONARQUBE UP COMMAND	14
9.1.2	CREATING A PROJECT IN SONARQUBE	14
9.2.1	MOIN MASTER WEB-APPLICATION SCAN COMMAND	15
9.2.2	MOIN MASTER SCANNING SUCCESS	15
9.2.3	SONARQUBE MOIN MASTER DASHBOARD	16
9.2.4	MOIN MASTER OVERALL REPORT	16
9.2.5	MOIN MASTER GRAPH REPORT	16
9.2.6	MOIN MASTER ISSUES IDENTIFIED	17
9.3.1	HACKAZON MASTER WEBAPPLICATION SCAN COMMAND	17
9.3.2	SCANNING HACKAZON MASTER	18
9.3.3	HACKAZON MASTER SCAN SUCCESS	18
9.3.4	HACKAZON OVERVIEW DASHBOARD	19
9.3.5	HACKAZON ISSUE IDENTIFIED	19
9.4.1	MOIN MASTER SEMGREP SCAN	20
9.4.2	MOIN MASTER SEMGREP SCAN ANALYSIS	20
9.4.3	MOIN MASTER SEMGREP SCAN FINDINGS	21
9.5.1	HACKAZON SEMGREP SCAN	21
9.5.2	HACKAZON SCAN ANALYSIS	22
9.5.3	HACKAZON SEMGREP SCAN FINDING AND SUCCESS	23

STATIC APPLICATION SECURITY TESTING (SAST) REPORT

1. MANAGEMENT OVERVIEW

This security audit utilized two popular open-source Static Application Security Testing (SAST) tools such as SonarQube and Semgrep which is used to perform scans on two web applications like Moin Master and Hackazon. Both the scans detected several security vulnerabilities across a spectrum of severity levels by providing crucial visibility into the security posture of the two applications. The primary objective of this assessment was to identify security vulnerabilities before deployment to production environments so that remediation can be done during the development cycle when fixes are cheapest. Both applications contained critical security vulnerabilities that must be remedied immediately.

2. SUMMARY OF KEY FINDINGS:

MOIN MASTER APPLICATION:

- **SonarQube:** Identified 11 bugs, 13 vulnerabilities, and 148 code smells with a Security Rating of 'E' (worst) referring to severe security vulnerabilities
- **Semgrep:** Identified a few security issues like unvalidated file inclusion vulnerabilities, potential SQL injection vulnerabilities, insecure validation practices, and inadequate input sanitization

HACKAZON APPLICATION:

- **SonarQube:** Identified 204 bugs, 121 vulnerabilities, and 2,062 code smells with reliability and security ratings of 'D', indicating severe security vulnerabilities
- **Semgrep:** Flagged many key security vulnerabilities, such as several SQL injection bugs, cross-site scripting (XSS) threats, insecure authentication management, poor access controls, and insecure session management

The results show that both applications need a lot of security remediation to be ready for production use. Without remediating these vulnerabilities, the applications pose substantial threat to organizational data and systems.

3. TOOLS AND SETUP

3.1 SONARQUBE

SonarQube is a cutting-edge open-source platform that was created for continuous code quality and security scanning and improvement. It performs automated static code analysis to detect bugs, smells, and security vulnerabilities in 27+ programming languages. SonarQube adheres to the rule of client-server architecture, with its server component providing an end-to-end web-based dashboard to display code quality metrics and narrative vulnerability reports. SonarQube review is controlled through a ruleset that can be customized to fit organizational requirements. The platform categorizes issues by severity (Blocker,

Critical, Major, Minor, Info) and by type (Bug, Vulnerability, Code Smell), allowing teams to maximize remediation efforts.

3.1.1 Installation Steps

Prerequisites:

- Install Docker and Docker Compose on your machine.
- Ensure your system meets the minimum requirements (2GB+ RAM).

Create **docker-compose.yml**:

The configuration I used for running the SonarQube with PostgreSQL:

Yaml file

```
version: '3.8'
```

```
services:
```

```
  sonarqube:
```

```
    image: sonarqube:community
```

```
    container_name: sonarqube
```

```
    ports:
```

```
      - "9000:9000"
```

```
    environment:
```

```
      - SONAR_JDBC_URL=jdbc:postgresql://db:5432/sonar
```

```
      - SONAR_JDBC_USERNAME=sonar
```

```
      - SONAR_JDBC_PASSWORD=sonar
```

```
    volumes:
```

```
      - sonarqube_data:/opt/sonarqube/data
```

```
      - sonarqube_extensions:/opt/sonarqube/extensions
```

```
      - sonarqube_logs:/opt/sonarqube/logs
```

```
    depends_on:
```

```
      - db
```

```
  db:
```

```
    image: postgres:13
```

```
    container_name: sonarqube_db
```

environment:

- POSTGRES_USER=sonar
- POSTGRES_PASSWORD=sonar
- POSTGRES_DB=sonar

volumes:

- postgresql_data:/var/lib/postgresql/data

volumes:

sonarqube_data:

sonarqube_extensions:

sonarqube_logs:

postgresql_data:version: '3.8'

services:

sonarqube:

image: sonarqube:community

container_name: sonarqube

ports:

- "9000:9000"

environment:

- SONAR_JDBC_URL=jdbc:postgresql://db:5432/sonar
- SONAR_JDBC_USERNAME=sonar
- SONAR_JDBC_PASSWORD=sonar

volumes:

- sonarqube_data:/opt/sonarqube/data
- sonarqube_extensions:/opt/sonarqube/extensions
- sonarqube_logs:/opt/sonarqube/logs

depends_on:

- db

db:

image: postgres:13

container_name: sonarqube_db

environment:

- POSTGRES_USER=sonar
- POSTGRES_PASSWORD=sonar
- POSTGRES_DB=sonar

volumes:

- postgresql_data:/var/lib/postgresql/data

volumes:

sonarqube_data:

sonarqube_extensions:

sonarqube_logs:

postgresql_data:

3.1.2 Start SonarQube:

When I Run the following command in the directory containing the docker-compose.yml It will start the sonarqube and the PostgreSQL database in detached mode

docker-compose up -d

3.1.3 Access SonarQube Web UI:

- Need to go in the URL : <http://localhost:9000>
- Default credentials: admin/admin
- Then we need to change our password

3.1.4 Install SonarScanner:

- Download and set up SonarScanner from the official documentation.
- Add the bin directory of SonarScanner to your system's PATH.

3.2 SEMGREP

Semgrep (Semantic Grep) is a light-weight, open-source, fast bug finding, vulnerability detection, and code standard enforcer static analysis tool. Semgrep was created by r2c (Return to Corporation) using a pattern-matching technique and special syntax that coalesces the convenience of grep with semantic code knowledge.

Unlike the elaborate configuration of typical static analysis tools, Semgrep is designed to be simple yet robust for programmers yet high-capability for security engineers. Semgrep supports over 30 languages, including JavaScript, Python, Java, Go, C, C++, and PHP.

Semgrep's rule system is specially potent, enabling the definition of custom rules in a YAML-based syntax more accessible than the usual static analysis rule definition. Semgrep Registry features thousands of pre-defined rules for standard security vulnerabilities, best practices, and framework-specific problems.

3.2.1 Installation Steps:

Prerequisites:

- We need Python 3.7 or higher version

Installation:

- Using the command **pip install semgrep** we can install the semgrep and check the version using **semgrep --version** to check the version of the semgrep

3.2.2 Basic Scanning:

- Run the Semgrep with the auto-configuration to use rules relevant to your codebase by going to the directory
semgrep --config=auto

Semgrep's approach focuses on precision, with an emphasis on minimizing false positives while catching real security issues. Its rule language is designed to be simple enough for developers to understand and extend, making it an effective tool for security-focused code reviews.

4. APPLICATION OVERVIEW AND ANALYSIS

4.1 MOIN MASTER

Moin Master is an advanced implementation of MoinMoin, a Python-based wiki engine. Based on the scan results and visible codebase structure, the application consists of significantly more than 2,000 lines of code with a complex architecture handling content management, user authentication, data storage, and web-based interactions.

Key Characteristics:

The Moin Master application is developed primarily in Python, supported by JavaScript for frontend functionality. It is developed based on the MVC (Model-View-Controller) architectural pattern, which supports a well-structured method of developing web applications. The heart of the application lies in its capability of wiki content management, along with user authentication, access control, and a plugin system to extend functionality. For the storage of data it comes with an SQL database backend built in likely SQLite or PostgreSQL based on the vulnerabilities identified during the analysis. The frontend uses HTML templates adorned with CSS and enriched with JavaScript interactions to present a dynamic user interface. User authentication is handled by an in-house developed system that has the good session management. The codebase is also a modular with an individual modules that addressing specific functionalities such as authentication and user management, content storage and retrieval, templating and rendering, plugin architecture, administration interface, search functionality, and access control and permissions. SonarQube has identified as a very high number of code smells (148) which implying potentially technical debt and maintenance in the codebase. These findings show that even though the application is feature-heavy always there might be some parts that require refactoring for increased code quality and maintainability over time.

4.2 HACKAZON

Hackazon is a very vulnerable web application for training and security testing. It also simulates an e-commerce website with the various functionalities like common to modern web applications. The application consists of more than 2,000 lines of code and exhibits numerous security vulnerabilities by design.

Key Characteristics:

It is developed using the PHP while JavaScript provides frontend features. The application is built on a layered architecture design which splits the application into distinct layers to enhance the modularity and maintainability. This applications core functionality includes the user registration and login to complete product catalog, management of shopping carts, payment handling, and review by the users. For database integration, it depends on a MySQL database with extensive Object-Relational Mapping (ORM) interactions to facilitate handling of data. The frontend employs HTML augmented with CSS frameworks for styling and extensive JavaScript functionality to provide a rich user interface. The user authentication system manages registration and login of users by session-based management.

The codebase is modular in structure with different parts handling some functionalities: user management and authentication, product inventory and catalog, shopping cart and order handling, payment processing, admin dashboard operations, review and ratings, and search. SonarQube detected an astronomically huge number of code smells (2,062) and bugs (204), which means that the application has been developed intentionally with

deliberate flaws for security testing. This makes Hackazon an ideal candidate for Static Application Security Testing (SAST) evaluation where developers and security professionals can identify and correct bugs in a safe environment.

5. REPORTED ISSUES

5.1 SONARQUBE FINDINGS - MOIN MASTER

The Moin Master application contains many severe vulnerabilities and code quality problems that seriously undermine its security and maintainability. SQL Injection vulnerabilities are prevalent with the users input is directly inserted into SQL queries without any parameterization. Findings the query ``query = "SELECT * FROM users WHERE username=" + username + "'"`` exposes the application to attackers who can manipulate inputs to inject arbitrary SQL commands. This attack is comprised of illegal data access, data alteration, or even full database breach.

The application further employs the bad cryptographic implementations, with very old algorithms like MD5 and SHA-1 to store sensitive data like passwords. The instance ``password_hash = md5(password)`` highlights this vulnerability, wherein modern computer processing power can just decrypt these hashes, leading to password exposure and illegal account entry. And another important weakness is a Cross-Site Scripting (XSS) due to lack of proper encoding of the users data before rendering the HTML templates. A good example is a templates using directly user-provided, unsanitized data, allowing attackers to inject malicious script which can be utilized to hijack sessions, steal information, or perform malicious operations on behalf of users.

Insecure Direct Object References were identified where the user-related data is accessed based only on the user-provided identifiers without the checking for authorization. The vulnerable call ``get_user_data(user_id)`` is how the attackers can tamper with the identifiers to grab an unauthorized data and displaying sensitive user data. Besides the above listed vulnerability this application also suffers from bugs such as null pointer dereferences (`user.getProfile().getSettings()`), resource leaks (absence of ``finally`` blocks to free resources) and logic issues in conditional expressions (`if (isAdmin || isAuthorized)` instead of `if (isAdmin && isAuthorized)`) all causing the potential crashes, instability and unauthorised access.

Moreover, the app indicates code smells of technical debt. These include extraneous code sections (e.g., redundant authentication code in multiple places), poor exception handling (``try { . } catch (Exception e) { log(e) }``) and overly complex methods (functions with over 100 lines and numerous nested conditionals). These issues increase the maintainability, testability, and security efforts of the codebase, leading to a greater likelihood of bugs and vulnerabilities. Together all these findings portend the call for urgent remediation measures for improving the safety and ease of maintenance of the Moin Master application.

5.2 SEMGREP FINDINGS - MOIN MASTER

The application carries the several severe weaknesses with severe security risks. There are some unvalidated file inclusion flaws since dynamic file inclusion based on user input isn't properly sanitized. The line of code `$_GET['module']. 'php'` can be exploited by remote attackers to cause malicious scripts from being included. This could lead to remote code execution and full system compromise. In addition, SQL injection attacks are present wherever user input is added to database queries without proper sanitization, for example in the example code `cursor.execute("SELECT * FROM users WHERE name LIKE '%" + search_term + "%'")`. This exposes the application to attack through the injection of malicious SQL code by the attacker that can be resulting in unauthorized access to data, data tampering or system compromise.

Moreover, the application employs weak password storage, weak hashing algorithms, or plain-text the storage of passwords with a good example for this is a hash `=hashlib.md5(password.encode()).hexdigest()` which makes compromised password hashes reversible it can lead to a higher chance of accounts being compromised and unauthorized access to other systems due to password reuse. Lastly, Cross-Site Request Forgery (CSRF) vulnerabilities are present because CSRF tokens are not being passed with form submissions. Without tokens for authentication, attackers have the ability to trick authenticated users into performing unwanted actions, and unauthorized actions will be performed on their behalf. These vulnerabilities overall highlight the need for immediate rectification in a bid to secure the application and its users from potential misuse.

5.3 SONARQUBE FINDINGS - HACKAZON

The Hackazon application contains a significant number of critical and high-severity vulnerabilities of totaling 121. The SQL Injection vulnerabilities are widespread with the users input being directly incorporated into the SQL queries without any proper sanitization. For instance the `"query = "SELECT * FROM products WHERE category_id = " . $_GET['category'];"` exposes the application to the risk of the attackers executing the arbitrary SQL commands which are leading to unauthorized access to sensitive data, data manipulation or complete database compromise. Additionally the Cross-Site Scripting (XSS) issues are arise due to the user input being rendered directly into HTML without encoding in `echo "<div>" . $_POST['comment'] . "</div>"`; This allows the attackers to execute malicious JavaScript in users browsers to the facilitating session hijacking, credential theft, or malicious actions performed with user privileges.

The application also suffers from the Cross-Site Request Forgery (CSRF) weaknesses, where the forms of lack CSRF tokens to validate request origins. For example, form submissions that change state without unique tokens allow attackers to trick users into performing unintended actions, resulting in unauthorized state changes or transactions. Furthermore, insecure session management is evident, with

session tokens exhibiting insufficient entropy or improper handling. The sessions not being regenerated after the privilege level changes increase the risk of session prediction or hijacking which enabling the unauthorized access to user accounts. Lastly the inadequate access controls are present within the missing or insufficient authorization checks. Admin functions accessible without proper verification of user roles expose the application to privilege escalation risks which allows to unauthorized access to administrative functionality.

In addition to these vulnerabilities, the application has 204 bugs, including runtime exceptions caused by unhandled exceptions that could crash the application, memory leaks due to improperly deallocated objects, and logical errors in business logic, such as incorrect discount calculations, which could lead to financial losses or incorrect business operations. The code bases also exhibits the 2062 code smells is the indicative of significant technical debt. These includes the duplicate code segments and complex methods with high cognitive complexity and inconsistent error handling approaches. Such issues make the code very difficult to maintain and increases the likelihood of the bugs and security problems which result in an inconsistent user experience while potentially leaking the sensitive information. Collectively, these findings highlight the need for comprehensive remediation efforts to improve the security and reliability of this application.

5.4 SEMGREP FINDINGS - HACKAZON

The Hackazon application exhibits several critical and high-severity security issues that pose significant risks to its integrity and confidentiality. Because from the input is frequently included into database queries without any adequate sanitization and the SQL Injection vulnerabilities are common. `sql = "SELECT * FROM users WHERE username = '{$_POST['username']}' AND password = '{$_POST['password']}'";` for example it enables the attackers to get around authentication or access data that isn't authorized which might result in a total breach of the database contents.

Additionally, the display of an unclean user-generated material in HTML output creates the Cross-Site Scripting (XSS) vulnerabilities. For instance the echo `"<h2>Results for: ". $_GET['search']. "</h2>";` allows the attackers to insert malicious JavaScript and making it easier to steal session information or sensitive user data. Another worrying problem is an insecure file uploads where the malicious scripts or executables are uploaded and not sufficiently validated and the attackers may be able to execute them remotely on the server.

In addition, the program has hardcoded credentials such as `db = new Database('localhost', 'root', 'password123', 'hackazon');`. Sensitive database credentials are exposed in the event that the source code is leaked and allowing for illegal database access. Additionally, there are some vulnerabilities for the command injection when the user input is directly incorporated into system operations as the demonstrated in `system("ping ". $_POST['server']);`. This enables

the attackers to carry out the arbitrary system commands in resulting in a total breach of the system.

The information disclosure vulnerabilities reveal specific error messages that divulge private information about the setup and structure of the program which includes the SQL queries and database table structures. These revelations make it very simpler to the attacker to find and take advantage of the vulnerabilities by giving them an insightful information for the future exploitation. All of these problems are point to the necessity of thorough cleanup procedures in order to protect the Hackazon application against potential threats.

6. COMPARATIVE ANALYSIS

6.1 SONARQUBES STRENGTH:

SonarQube provide the thorough code quality analysis that goes beyond the security issues which is offering a wide range of metrics like maintainability metrics, code coverage, and duplication percentage. For example it revealed that the Hackazon program had 148 days of technical debt and a 2.1percentage duplication rate which is providing the important information on the general state of the codebase. The teams can be resolved in both the security flaws and more general problem with the code quality with its comprehensive approach.

The tool is further improved by its outstanding reporting and visualization features which include a sophisticated dashboard with the visual representations of data related to code quality. The heat map that show the distribution of the vulnerabilities across the component are on one example they help management and stakeholders better understand and rank important issues. This feature ensures effective communication of complex data in an accessible format.

SonarQube's comprehensive classification system makes it as an easy to group problems according to their nature (bug, vulnerability, code smell) and severity (blocker, critical, major, minor, information). For instance it distinguish between the serious SQL injection flaws and minor style problems which allows the teams to concentrate their repair efforts on the most urgent issues first. Effectiveness in the resources allocation and prioritization are made possible by this fine classification.

The SonarQube provides a code quality rating system by assigning the letter grades (A-E) for key metrics like reliability, security, and maintainability. The 'E' grade given to Moin Master security rating serves as an example which is highlighting the serious problems that need to be addressed right once. These ratings provides a rapid and simple method for evaluating the code base in overall quality in a number of different areas.

By integrating the platform into a CI/CD pipelines the teams can measures the quality of indicators over the time and do its historical trend analysis. For example it can track the growth or decline of vulnerabilities, assisting enterprises in gauging the success of their security enhancement initiatives. Throughout the development lifetime, our longitudinal approach guarantees ongoing code quality monitoring and improvement.

6.2 SEMGREP STRENGTHS:

Semgrep excels at security-specific analysis which is specifically targeting the detection of security vulnerabilities with fewer false positives from non-security-related problems. It is for instance directly identifies the SQL injection patterns without being diverted to stylistic problems in offering an improved signal-to-noise ratio for security-related problems. It employs pattern-based detection which has a feature that enables it to accurately identify patterns of vulnerability across the codebase. This enables Semgrep to identify all occurrences of a specific vulnerability pattern within a single scan with the additional coverage of related problems.

Semgrep also has a lower false positive rate with more accurate detection of actual security vulnerabilities. It clearly distinguishes between legitimate SQL injection vulnerabilities and secure SQL use the time that would be wasted pursuing non-issues. In addition to this the semgrep gives remediation guidance that is clear with recommendations for specific fixes for issues found. For instance it advises the use of parameterized queries over string concatenation to remediate SQL injection vulnerabilities which results in quicker and more efficient remediation activities.

The second important benefit of Semgrep is that it is fast and lightweight. It uses less resource and takes much less time to finish scanning compared to tools such as SonarQube. For the Hackazon application, Semgrep took much less time to finish scanning, which would make it easier to be integrated into developers workflows such as pre-commit hooks and CI/CD pipelines. All these characteristics of Semgrep qualify it as an ideal tool for quick and targeted security testing in development.

6.3 OVERLAP IN FINDINGS:

6.3.1 Similar Findings:

Both Semgrep and SonarQube were able to detect the most severe security vulnerabilities in both application. They were able to detect the several occurrences of SQL Injection vulnerabilities where the user input is directly used in database queries without any parameterization. Both the tools also detected Cross-Site Scripting (XSS) problems which is a failure to sanitize user input adequately before displaying it in HTML output that would permit attackers to inject malicious scripts. In addition, weaknesses in authorization and authentication mechanisms were found, e.g., lack of proper verification of user privileges before granting access to sensitive functionality. These overlapping results bear witness to the effectiveness of both tools at identifying serious security vulnerabilities that would compromise the integrity and confidentiality of the applications.

6.3.2 Unique Findings by SonarQube:

SonarQube also reported some additional generic code quality problems that reach beyond the security vulnerabilities such as highlighting considerable technical debt and the maintainability problems in both applications. For example the scan we picked up on complex methods, duplicated blocks of code, and ineffective exception handling which are all factors in making maintenance harder and security implementation less consistent. It is also picked up on the resource management problem such as database connections and file handle left open which can cause the resource exhaustion or application instability. Architecturally, SonarQube highlight issues such as extremely tight coupling of components, which would make it more difficult to maintain strong security boundaries. On top of this performance hotspots were identified such as inefficient database queries and duplicate calculation that could be an impact in the application performance under the load. In general these findings highlight the necessity of considerable remediation effort to tackled not just security weaknesses but also overall code quality and maintainability.

6.3.3 Unique Findings by Semgrep:

Semgrep had some specific strengths in how it scanned the applications. It provided more accurate vulnerability instances, providing detailed context such as exact line numbers and code paths for vulnerabilities like SQL injection vulnerabilities. It also performed well at framework-specific vulnerability detection, identifying issues particular to the frameworks being utilized, such as authentication bypasses. Semgrep also revealed business logic vulnerabilities, representing incorrect implementations of multi-step security flows that an attacker could leverage to their benefit. Finally, it identified security misconfigurations, such as insecure cookie flags and incorrect HTTP headers, that could result in potential vulnerabilities. These features make Semgrep a powerful tool for finding exact and subtle security vulnerabilities.

6.4 False Positives Analysis:

SonarQube tends to flag more potential false positives, particularly in the "Code Smell" category, where stylistic issues are sometimes flagged as security issues. This has the impact of diluting the priority given to actual serious security issues, making it harder for teams to properly prioritize remediation efforts.

Semgrep tends to have fewer false positives, with fewer critical security categories. It surpasses other tools in distinguishing true vulnerabilities, such as injection flaws, from secure code patterns, allowing for more efficient prioritization of remediation. This positions Semgrep as particularly ideal for targeted security scanning, where developers can concentrate their efforts on repairing genuine risks rather than minor or non-critical issues.

6.5 Tool Integration and Usability:

SonarQube is more appropriate for enterprise setups as it requires more sophisticated setup and configuration needs such as the use of a database server and more system resources. Although it offers a detailed analysis and accurate metrics it is also a difficult to integrate into individual developer workflows.

In contrast the Semgrep has easier installation and use with the minimal resource needs that enable it to be used in CI/CD pipelines. It is a ease of use for developers and appropriateness for pre-commit hooks enable fast, targeted security scanning in the development workflow, thus rendering it a perfect tool for developers looking for effective and time-reducing security testing.

7. CONCLUSION

For overall security and code quality analysis in a continuous integration setup, SonarQube has no equal in its detailed dashboard, large number of metrics, and overall quality view of the code. Its issue tracking over time and total quality view makes it a valuable tool for managing code in the long term. Semgrep, on the other hand, shines at targeted security vulnerability hunting, particularly where immediate feedback is paramount. Its high-performance configuration, reduced false positive rate, and security-oriented approach make Semgrep perfectly positioned to be incorporated within daily workflows for developers. Both the tools are best used by employing them both: use Semgrep for quicker, security-based scanning at dev time and pre-commit hooks, and SonarQube for deeper analysis during CI/CD pipelines and longer-term quality monitoring. This two-tool approach ensures optimal coverage of security vulnerabilities and addresses general code quality problems that indirectly affect the security posture of the application.

8. REFERENCES

- SonarQube Documentation. (2023). <https://docs.sonarqube.org/latest/>
- Semgrep Documentation. (2023). <https://semgrep.dev/docs/>

9. APPENDIX

9.1 APPENDIX 1 – SONARQUBE:

```
(kali㉿kali) - [~]
$ cd ~/sonarqube

(kali㉿kali) - [~/sonarqube]
$ sudo docker-compose up -d
[sudo] password for kali:
Starting sonarqube_db_1 ... done
Starting sonarqube_sonarqube_1 ... done

(kali㉿kali) - [~/sonarqube]
$
```

Fig 9.1.1 – Sonarqube up command

2 Run analysis on your project

What option best describes your project?

Maven

Gradle

.NET

Other (for JS, TS, Go, Python, PHP, ...)

What is your OS?

Linux

Windows

macOS

Download and unzip the Scanner for Linux

Visit the [official documentation of the Scanner](#) to download the latest version, and add the `bin` directory to the `PATH` environment.

Execute the Scanner

Running a SonarQube analysis is straightforward. You just need to execute the following commands in your project's folder.

```
sonar-scanner \
-Dsonar.projectKey=Assignment-Q3 \
-Dsonar.sources=. \
-Dsonar.host.url=http://localhost:9000 \
-Dsonar.token=sqp_5aec2531b8d2d1542b9abb0f1d3683e21d76b75d
```

Please visit the [official documentation of the Scanner](#) for more details.

Fig 9.1.2 – Creating a project in sonarqube

9.2 APPENDIX 2 : MOIN MASTER WEB APPLICATION – SONARQUBE

```
(kali@kali) ~ - /Downloads/sonar-scanner-7.0.2.4839-linux-x64/bin]
$ ./sonar-scanner
-Dsonar.projectKey=Assignment-Q1 \
-Dsonar.sources=/home/kali/Downloads/moin-master/ \
-Dsonar.host.url=http://localhost:9000 \
-Dsonar.token=sqp_9c57c1979ced095f2f23051fd8795d1a313a0935 \
-Dsonar.projectBaseDir=/home/kali/Downloads/moin-master
>
15:39:46.442 INFO Scanner configuration file: /home/kali/Downloads/sonar-scanner-7.0.2.4839
15:39:46.446 INFO Project root configuration file: NONE
15:39:46.461 INFO SonarScanner CLI 7.0.2.4839
15:39:46.464 INFO Java 17.0.13 Eclipse Adoptium (64-bit)
15:39:46.464 INFO Linux 6.12.13-amd64 amd64
15:39:46.494 INFO User cache: /home/kali/.sonar/cache
15:39:47.254 INFO JRE provisioning: os[linux], arch[x86_64]
15:39:47.351 INFO Communicating with SonarQube Community Build 25.2.0.102705
15:39:47.700 INFO Starting SonarScanner Engine...
15:39:47.703 INFO Java 17.0.13 Eclipse Adoptium (64-bit)
15:39:48.773 INFO Load global settings
15:39:48.890 INFO Load global settings (done) | time=118ms
15:39:48.898 INFO Server id: 243B8A4D-AZVYVT4kKTA_gpY1slqE
15:39:48.918 INFO Loading required plugins
15:39:48.922 INFO Load plugins index
15:39:48.943 INFO Load plugins index (done) | time=23ms
15:39:49.023 INFO Load/download plugins (done) | time=81ms
15:39:49.368 INFO Process project properties
15:39:49.382 INFO Process project properties (done) | time=14ms
15:39:49.397 INFO Project key: Assignment-Q1
15:39:49.401 INFO Base dir: /home/kali/Downloads/moin-master
15:39:49.405 INFO Working dir: /home/kali/Downloads/moin-master/.scannerwork
15:39:49.409 INFO Load project settings for component key: 'Assignment-Q1'
15:39:49.437 INFO Load project settings for component key: 'Assignment-Q1' (done) | time=24
15:39:49.500 INFO Load quality profiles
15:39:49.617 INFO Load quality profiles (done) | time=118ms
15:39:49.682 WARN SCM provider autodetection failed. Please use "sonar.scm.provider" to def
15:39:49.694 INFO Load active rules
15:39:50.195 INFO Load active rules (done) | time=500ms
15:39:50.203 INFO Load analysis cache
15:39:50.218 INFO Load analysis cache (404) | time=15ms
15:39:50.287 INFO Preprocessing files...
15:39:50.524 INFO 6 languages detected in 798 preprocessed files
```

Fig 9.2.1-MOIN MASTER WEB-APLLICATION SCAN COMMAND

```
15:40:28.874 INFO Miss the cache for 0 out of 0
15:40:28.874 INFO Sensor CSS Rules [javascript] (done) | time=1235ms
15:40:28.875 INFO Sensor CSS Metrics [javascript]
15:40:29.017 INFO Sensor IAC Docker Sensor [iac]
15:40:29.018 INFO 0 source files to be analyzed
15:40:29.061 INFO 0/0 source files have been analyzed
15:40:29.062 INFO Sensor IAC Docker Sensor [iac] (done) | time=47ms
15:40:29.062 INFO Sensor TextAndSecretsSensor [text]
15:40:29.063 INFO Available processors: 4
15:40:29.063 INFO Using 4 threads for analysis.
15:40:29.495 INFO The property "sonar.tests" is not set. To improve the analysis accuracy,
* The filename starts with "test"
* The filename contains "test." or "tests."
* Any directory in the file path is named: "doc", "docs", "test" or "tests"
* Any directory in the file path has a name ending in "test" or "tests"
15:40:29.516 INFO Using git CLI to retrieve untracked files
15:40:29.521 WARN Analyzing only language associated files, make sure to run the analysis i
15:40:29.690 INFO 465 source files to be analyzed
15:40:30.312 INFO 465/465 source files have been analyzed
15:40:30.313 INFO Sensor TextAndSecretsSensor [text] (done) | time=1252ms
15:40:30.334 INFO ----- Run sensors on project
15:40:30.556 INFO Sensor Zero Coverage Sensor
15:40:30.620 INFO Sensor Zero Coverage Sensor (done) | time=65ms
15:40:30.624 INFO SCM Publisher No SCM system was detected. You can use the 'sonar.scm.prov
15:40:30.682 INFO CPD Executor 60 files had no CPD blocks
15:40:30.682 INFO CPD Executor Calculating CPD for 382 files
15:40:30.899 INFO CPD Executor CPD calculation finished (done) | time=216ms
15:40:31.222 INFO Analysis report generated in 297ms, dir size=7.9 MB
15:40:32.893 INFO Analysis report compressed in 1669ms, zip size=4.4 MB
15:40:33.185 INFO Analysis report uploaded in 292ms
15:40:33.187 INFO ANALYSIS SUCCESSFUL, you can find the results at: http://localhost:9000/d
15:40:33.188 INFO Note that you will be able to access the updated dashboard once the serve
15:40:33.189 INFO More about the report processing at http://localhost:9000/api/ce/task?id=
15:40:33.438 INFO Analysis total time: 44.306 s
15:40:33.441 INFO SonarScanner Engine completed successfully
15:40:34.167 INFO EXECUTION SUCCESS
15:40:34.190 INFO Total time: 47.748s
```

Fig 9.2.2- MOIN MASTER SCANNING SUCCESS

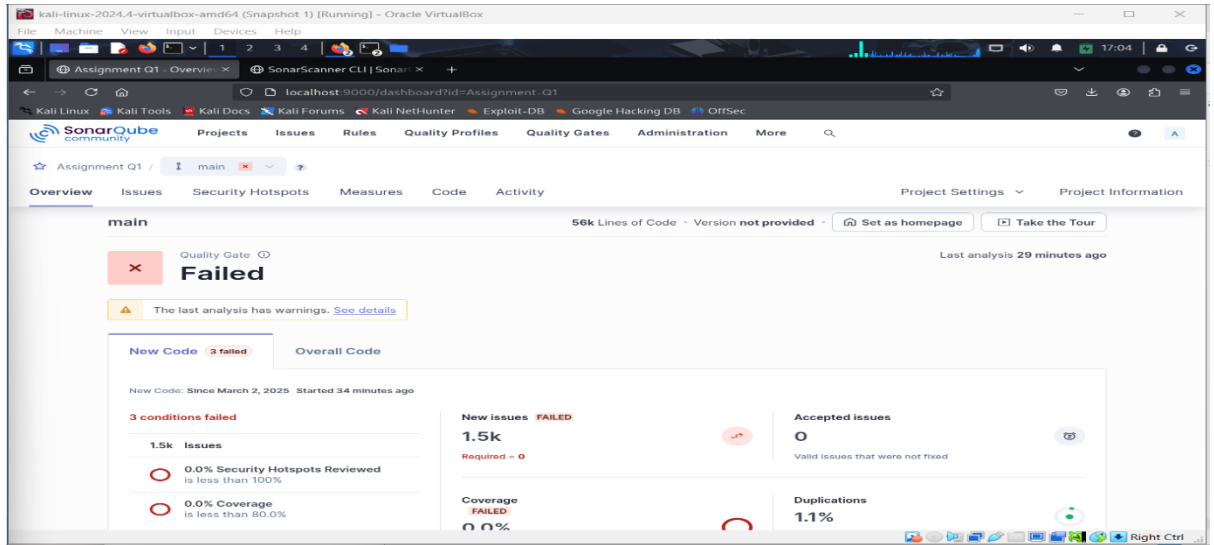


Fig 9.2.3 - SONARQUBE MOIN MASTER DASHBOARD

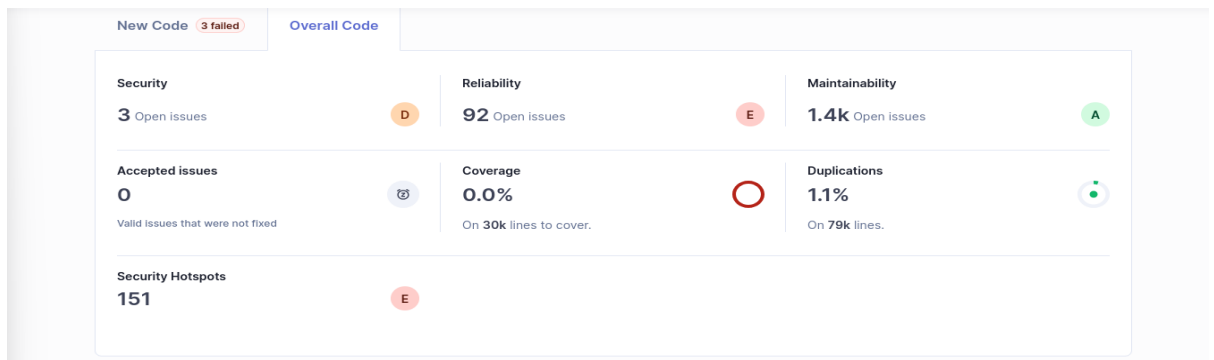


Fig 9.2.4 – MOIN MASTER OVERALL REPORT

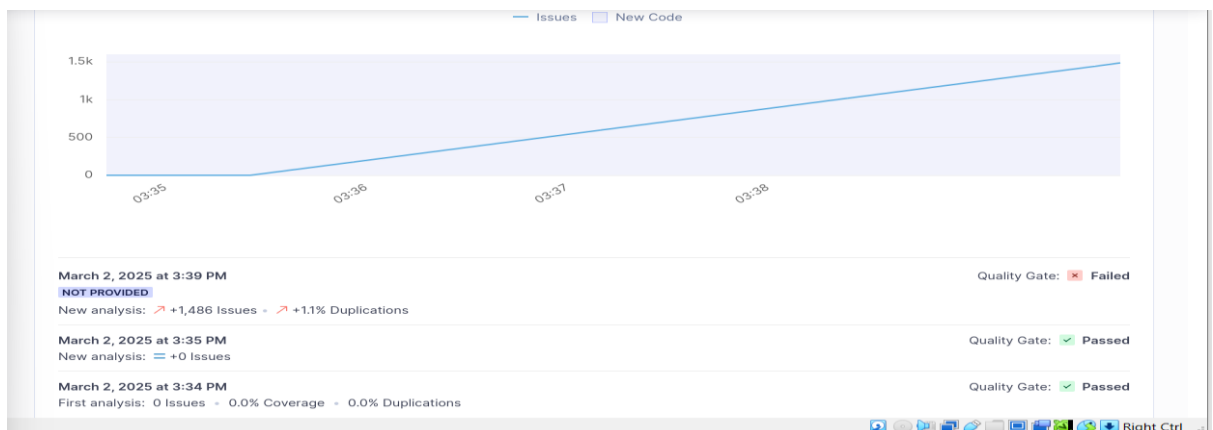


Fig 9.2.5 – MOIN MASTER GRAPH REPORT

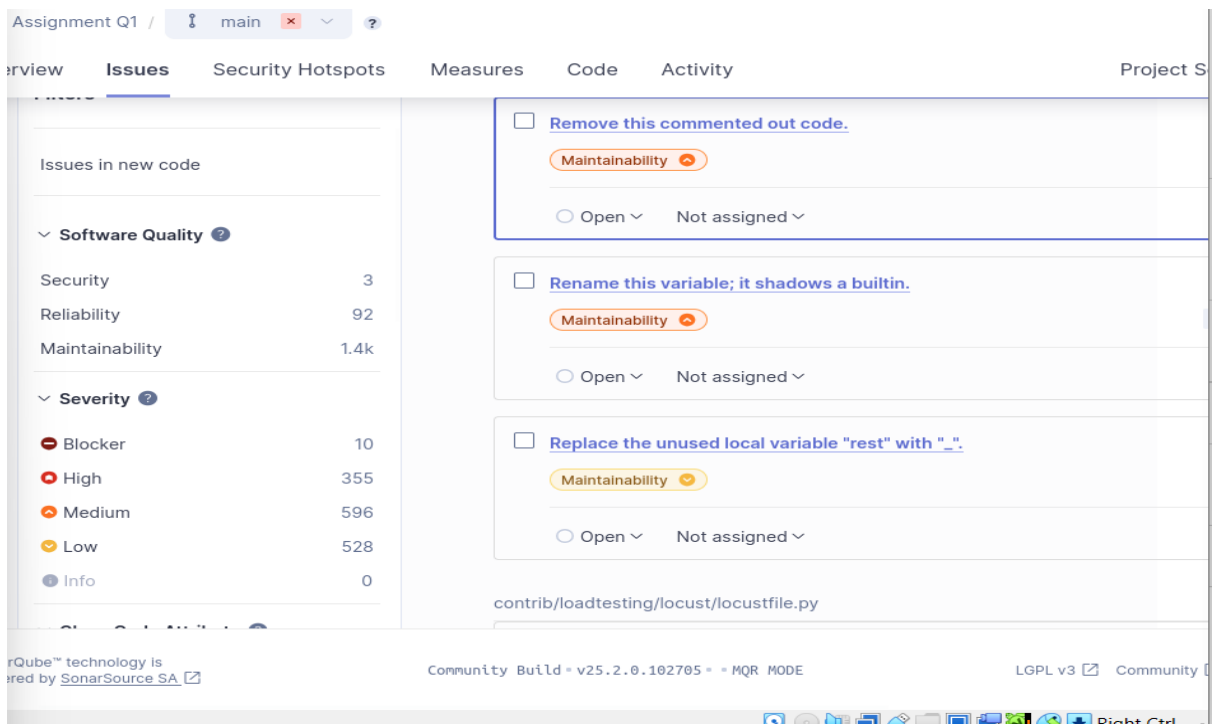


Fig 9.2.6 – MOIN MASTER ISSUES IDENTIFIED

9.3 APPENDIX 3 :HACKAZON MASTER WEB APPLICATION – SONARQUBE

```

(kali@kali) - [~/Downloads/sonar-scanner-7.0.2.4839-linux-x64/bin]
$ ./sonar-scanner \
-Dsonar.projectKey=Assignment-Q2 \
-Dsonar.sources=/home/kali/Downloads/hackazon-master/ \
-Dsonar.host.url=http://localhost:9000 \
-Dsonar.token=sqp_79c6dc9a04acf564810560122cfe6c13fffd7a77 \
-Dsonar.projectBaseDir=/home/kali/Downloads/hackazon-master/
7:12:41.728 INFO Scanner configuration file: /home/kali/Downloads/sonar-scanner-7.0.2.4839
7:12:41.733 INFO Project root configuration file: NONE
7:12:41.758 INFO SonarScanner CLI 7.0.2.4839
7:12:41.761 INFO Java 17.0.13 Eclipse Adoptium (64-bit)
7:12:41.762 INFO Linux 6.12.13-amd64 amd64
7:12:41.823 INFO User cache: /home/kali/.sonar/cache
7:12:42.727 INFO JRE provisioning: os[linux], arch[x86_64]
7:12:42.979 INFO Communicating with SonarQube Community Build 25.2.0.102705
7:12:43.396 INFO Starting SonarScanner Engine...
7:12:43.397 INFO Java 17.0.13 Eclipse Adoptium (64-bit)
7:12:44.735 INFO Load global settings
7:12:44.861 INFO Load global settings (done) | time=125ms
7:12:44.864 INFO Server id: 243B8A4D-AZVYVT4kKTA_gpY1slqE
7:12:44.881 INFO Loading required plugins
7:12:44.885 INFO Load plugins index
7:12:44.966 INFO Load plugins index (done) | time=81ms
7:12:44.967 INFO Load/download plugins
7:12:45.038 INFO Load/download plugins (done) | time=76ms
7:12:45.526 INFO Process project properties

```

Maintainability: 0, 10k, 23.2%, 216k lines

Fig 9.3.1 – HACKAZON MASTER WEBAPPLICATION SCAN COMMAND

```

17:15:10.133 INFO Some of the project files were automatically excluded because they looked
le detection by setting sonar.javascript.detectBundles=false
17:15:10.700 INFO 2490 files indexed
17:15:10.702 INFO Quality profile for css: Sonar way
17:15:10.702 INFO Quality profile for flex: Sonar way
17:15:10.703 INFO Quality profile for js: Sonar way
17:15:10.704 INFO Quality profile for json: Sonar way
17:15:10.704 INFO Quality profile for php: Sonar way
17:15:10.704 INFO Quality profile for py: Sonar way
17:15:10.704 INFO Quality profile for web: Sonar way
17:15:10.708 INFO Quality profile for xml: Sonar way
17:15:10.708 INFO Quality profile for yaml: Sonar way
17:15:10.708 INFO ----- Run sensors on module Assignment-Q2
17:15:10.762 INFO Load metrics repository
17:15:10.789 INFO Load metrics repository (done) | time=27ms
17:15:11.667 INFO Sensor Python Sensor [python]
17:15:11.677 WARN Your code is analyzed as compatible with all Python 3 versions by default
ia the parameter "sonar.python.version"
17:15:11.705 INFO Starting global symbols computation
17:15:11.715 INFO 2 source files to be analyzed
17:15:12.123 INFO 2/2 source files have been analyzed
17:15:12.152 INFO Starting rules execution
17:15:12.156 INFO 2 source files to be analyzed
17:15:12.636 INFO 2/2 source files have been analyzed
17:15:12.640 INFO The Python analyzer was able to leverage cached data from previous analys
17:15:12.648 INFO Sensor Python Sensor [python] (done) | time=971ms
17:15:12.649 INFO Sensor Cobertura Sensor for Python coverage [python]
17:15:12.672 INFO Sensor Cobertura Sensor for Python coverage [python] (done) | time=34ms
17:15:12.673 INFO Sensor PythonXUnitSensor [python]
17:15:12.715 INFO Sensor PythonXUnitSensor [python] (done) | time=42ms
17:15:12.716 INFO Sensor HTML [web]
17:15:13.305 WARN Invalid character encountered in file /home/kali/Downloads/hackazon-maste
e 6 for encoding UTF-8. Please fix file content or configure the encoding to be used using p
17:15:14.034 INFO Sensor HTML [web] (done) | time=1318ms
17:15:14.037 INFO Sensor Flex [flex]
17:15:14.042 INFO 180 source files to be analyzed

```

Fig 9.3.2 – SCANNING HACKAZON MASTER

```

17:16:27.020 INFO Using 6 threads for analysis.
17:16:27.369 INFO The property "sonar.tests" is not set. To improve the analysis accuracy,
* The filename starts with "test"
* The filename contains "test." or "tests."
* Any directory in the file path is named: "doc", "docs", "test" or "tests"
* Any directory in the file path has a name ending in "test" or "tests"
17:16:27.406 INFO Using git CLI to retrieve untracked files
17:16:27.414 WARN Analyzing only language associated files, make sure to run the analysis i
17:16:27.659 INFO 1267 source files to be analyzed
17:16:28.588 INFO 1267/1267 source files have been analyzed
17:16:28.591 INFO Sensor TextAndSecretsSensor [text] (done) | time=1593ms
17:16:28.597 INFO ----- Run sensors on project
17:16:28.790 INFO Sensor Zero Coverage Sensor
17:16:28.876 INFO Sensor Zero Coverage Sensor (done) | time=86ms
17:16:28.877 INFO SCM Publisher No SCM system was detected. You can use the 'sonar.scm.prov
17:16:28.931 INFO CPD Executor 196 files had no CPD blocks
17:16:28.931 INFO CPD Executor Calculating CPD for 559 files
17:16:29.218 INFO CPD Executor CPD calculation finished (done) | time=286ms
17:16:29.764 INFO Analysis report generated in 509ms, dir size=18.5 MB
17:16:30.633 INFO Analysis report compressed in 866ms, zip size=6.9 MB
17:16:31.265 INFO Analysis report uploaded in 632ms
17:16:31.267 INFO ANALYSIS SUCCESSFUL, you can find the results at: http://localhost:9000/d
17:16:31.267 INFO Note that you will be able to access the updated dashboard once the serve
17:16:31.268 INFO More about the report processing at http://localhost:9000/api/ce/task?id=
17:16:31.493 INFO Analysis total time: 1:23.758 s
17:16:31.496 INFO SonarScanner Engine completed successfully
17:16:31.867 INFO EXECUTION SUCCESS
17:16:31.869 INFO Total time: 1:26.998s
(kali@kali) - [~/Downloads/sonar-scanner-7.0.2.4839-linux-x64/bin]
$

```

Fig 9.3.3 – HACKAZON MASTER SCAN SUCCESS

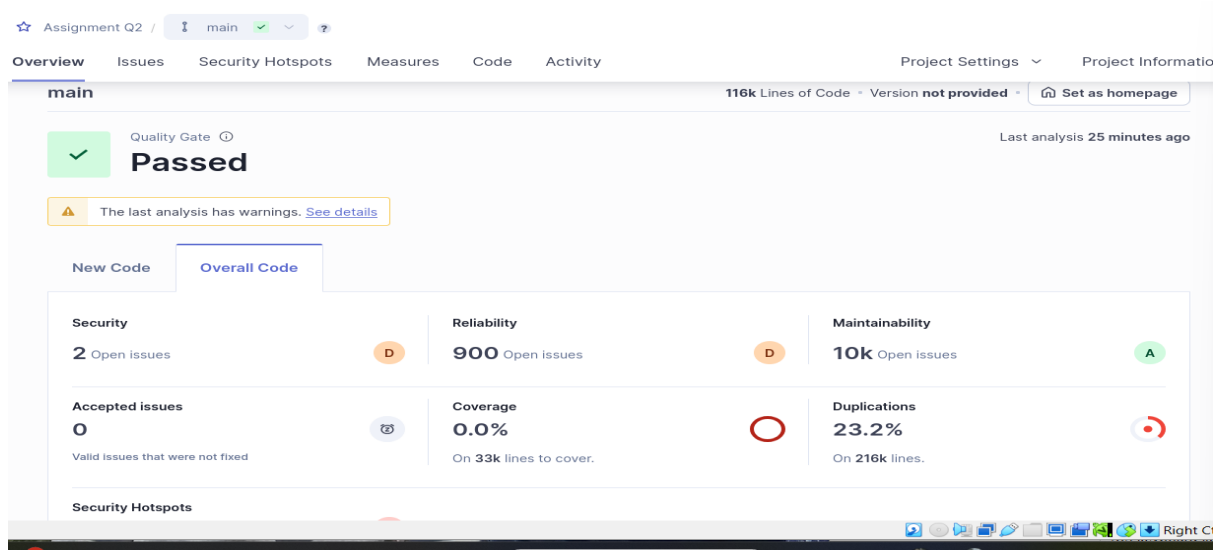


Fig 9.3.4 – HACKAZON OVERVIEW DASHBOARD

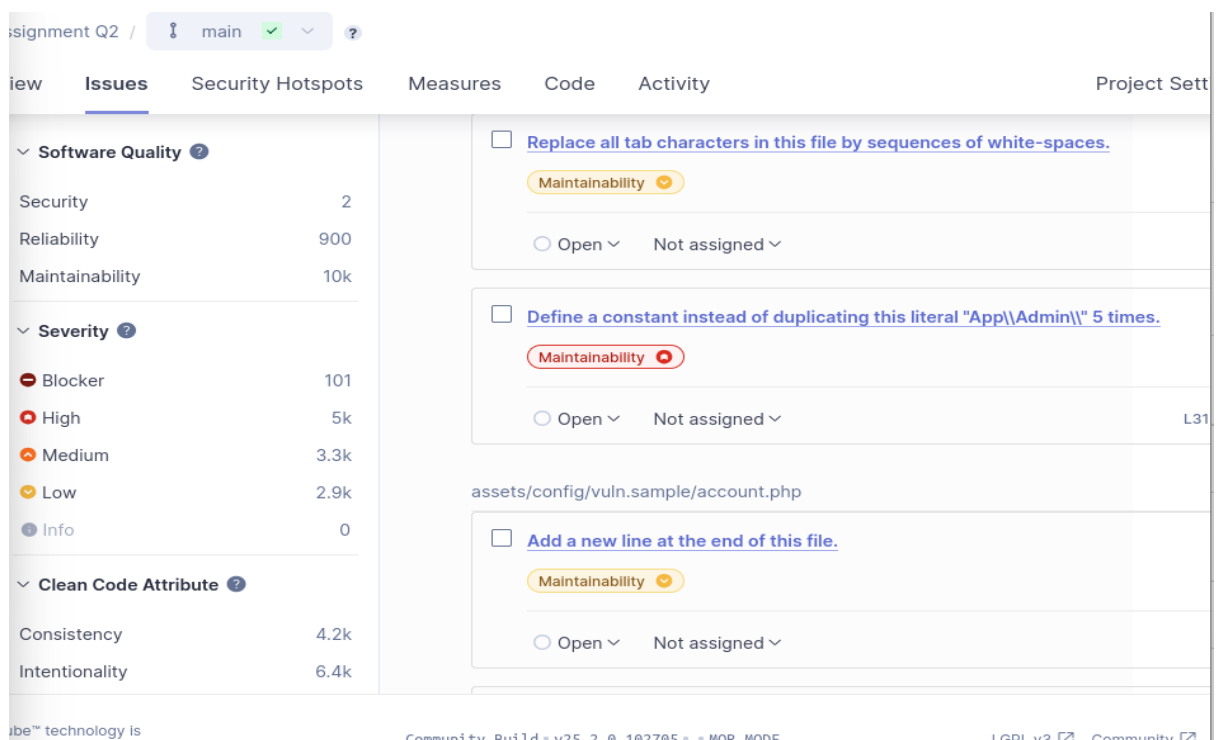
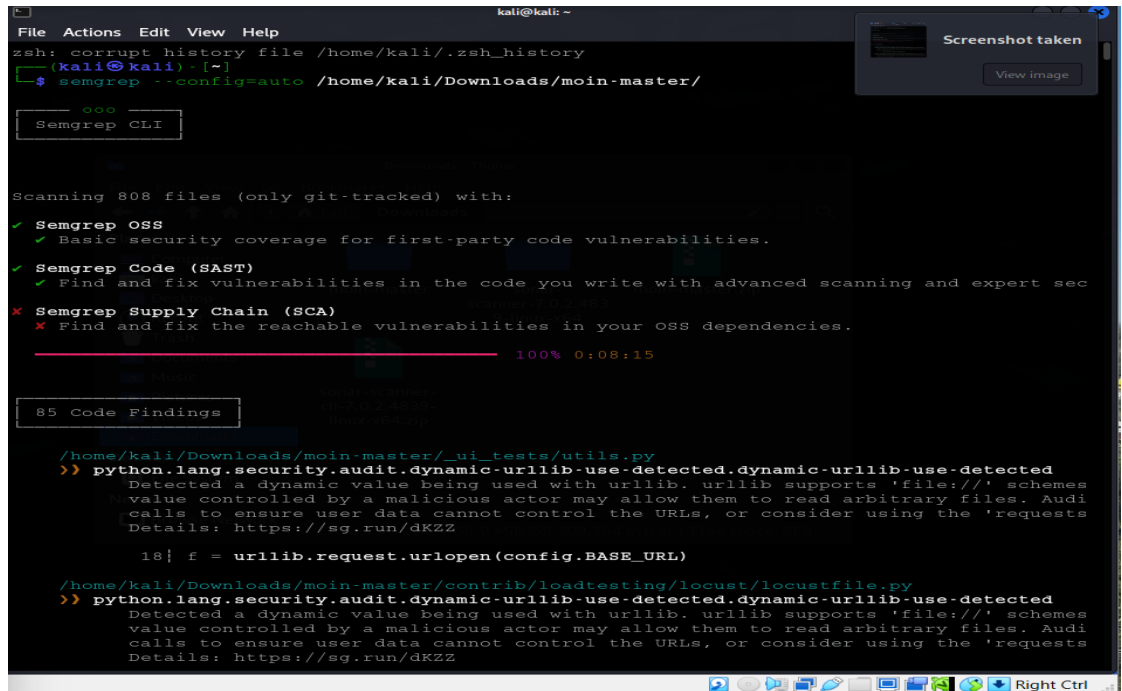


Fig 9.3.5 – HACKAZON ISSUE IDENTIFIED

9.4 APPENDIX 4 : MOIN MASTER SEMGREP SCAN



```
File Actions Edit View Help
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali) ~
$ semgrep --config=auto /home/kali/Downloads/moin-master/

Semgrep CLI

Scanning 808 files (only git-tracked) with:
✓ Semgrep OSS
  ✓ Basic security coverage for first-party code vulnerabilities.
✓ Semgrep Code (SAST)
  ✓ Find and fix vulnerabilities in the code you write with advanced scanning and expert sec
✗ Semgrep Supply Chain (SCA)
  ✗ Find and fix the reachable vulnerabilities in your OSS dependencies.

100% 0:08:15

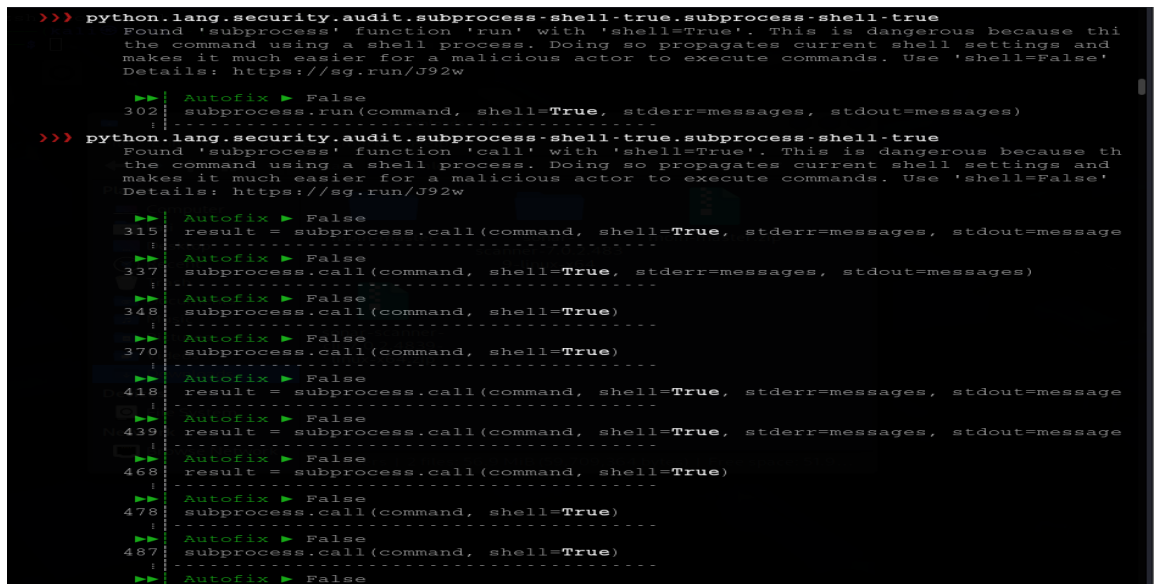
85 Code Findings

/home/kali/Downloads/moin-master/_ui_tests/utils.py
>> python.lang.security.audit.dynamic-urllib-use-detected.dynamic-urllib-use-detected
Detected a dynamic value being used with urllib. urllib supports 'file://' schemes
value controlled by a malicious actor may allow them to read arbitrary files. Audi
calls to ensure user data cannot control the URLs, or consider using the 'requests
Details: https://sg.run/dKZZ

18| f = urllib.request.urlopen(config.BASE_URL)

/home/kali/Downloads/moin-master/contrib/loadtesting/locust/locustfile.py
>> python.lang.security.audit.dynamic-urllib-use-detected.dynamic-urllib-use-detected
Detected a dynamic value being used with urllib. urllib supports 'file://' schemes
value controlled by a malicious actor may allow them to read arbitrary files. Audi
calls to ensure user data cannot control the URLs, or consider using the 'requests
Details: https://sg.run/dKZZ
```

Fig 9.4.1 – MOIN MASTER SEMGREP SCAN



```
>>> python.lang.security.audit.subprocess-shell-true.subprocess-shell-true
Found 'subprocess' function 'run' with 'shell=True'. This is dangerous because thi
the command using a shell process. Doing so propagates current shell settings and
makes it much easier for a malicious actor to execute commands. Use 'shell=False'
Details: https://sg.run/J92w

Autofix ▶ False
302 subprocess.run(command, shell=True, stderr=messages, stdout=messages)

>>> python.lang.security.audit.subprocess-shell-true.subprocess-shell-true
Found 'subprocess' function 'call' with 'shell=True'. This is dangerous because th
the command using a shell process. Doing so propagates current shell settings and
makes it much easier for a malicious actor to execute commands. Use 'shell=False'
Details: https://sg.run/J92w

Autofix ▶ False
315 result = subprocess.call(command, shell=True, stderr=messages, stdout=message
Autofix ▶ False
337 subprocess.call(command, shell=True, stderr=messages, stdout=messages)
Autofix ▶ False
348 subprocess.call(command, shell=True)
Autofix ▶ False
370 subprocess.call(command, shell=True)
Autofix ▶ False
418 result = subprocess.call(command, shell=True, stderr=messages, stdout=message
Autofix ▶ False
439 result = subprocess.call(command, shell=True, stderr=messages, stdout=message
Autofix ▶ False
468 result = subprocess.call(command, shell=True)
Autofix ▶ False
478 subprocess.call(command, shell=True)
Autofix ▶ False
487 subprocess.call(command, shell=True)
Autofix ▶ False
```

Fig 9.4.2 – MOIN MASTER SEMGREP SCAN ANALYSIS

```
kali@kali: ~  
File Actions Edit View Help  
Autofix ▶ if django.contrib.auth.password_validation.validate_password(password)  
self.set_password(password)  
364 self.set_password(password)  
-----  
Autofix ▶ if django.contrib.auth.password_validation.validate_password(newpass)  
self.set_password(newpass)  
844 self.set_password(newpass)  
  
>> python.flask.security.unescaped-template-extension.unescaped-template-extension  
Flask does not automatically escape Jinja templates unless they have .html, .htm,  
extensions. This could lead to XSS attacks. Use .html, .htm, .xml, or .xhtml for y  
extensions. See https://flask.palletsprojects.com/en/1.1.x/templating/#jinja-setup  
information.  
Details: https://sg.run/xlRg  
  
862 text = render_template("mail/password_recovery.txt", link=link)  
-----  
872 text = render_template("mail/account_verification.txt", link=link)  
  
/home/kali/Downloads/moin-master/src/moin/utils/pysupport.py  
>> python.lang.security.audit.non-literal-import.non-literal-import  
Untrusted user input in 'importlib.import_module()' function allows an attacker to  
code. Avoid dynamic values in 'importlib.import_module()' or use a whitelist to pr  
untrusted code.  
Details: https://sg.run/y6Jk  
  
37| importlib.import_module(module)  
  
Scan Summary  
Some files were skipped or only partially analyzed.  
Partially scanned: 54 files only partially analyzed due to parsing or internal Semgrep err  
Scan skipped: 1 files matching .semgrepignore patterns  
For a full list of skipped files, run semgrep with the --verbose flag.  
  
Ran 1281 rules on 807 files: 85 findings.  
Too many findings? Try Semgrep Pro for more powerful queries and less noise.  
See https://sg.run/false-positives.
```

Fig 9.4.3 – MOIN MASTER SEMGREP SCAN FINDINGS

9.5 APPENDIX 5 HACKAZON SEMGREP SCAN

```
(kali@kali) - [~]  
$  
semgrep --config=auto /home/kali/Downloads/hackazon-master/  
  
Semgrep CLI  
Scanning 5955 files (only git-tracked) with:  
✓ Semgrep OSS  
✓ Basic security coverage for first-party code vulnerabilities.  
✓ Semgrep Code (SAST)  
✓ Find and fix vulnerabilities in the code you write with advanced scanning and expert security  
✗ Semgrep Supply Chain (SCA)  
✗ Find and fix the reachable vulnerabilities in your OSS dependencies.
```

Fig 9.5.1 HACKAZON SEMGREP SCAN

```
>>> php.lang.security.exec-use.exec-use
Executing non-constant commands. This can lead to command injection.
Details: https://sg.run/5Q1j

105| exec('type ' . $path, $content);
107| exec('cat ' . $path, $content);

/home/kali/Downloads/hackazon-master/classes/App/Controller/Checkout.php
>> php.symfony.security.audit.symfony-non-literal-redirect.symfony-non-literal-redirect
The 'redirect()' method does not check its destination in any way. If you redirect to a URL provided
by end-users, your application may be open to the unvalidated redirects security vulnerability.
Consider using literal values or an allowlist to validate URLs.
Details: https://sg.run/4ey5

279| $this->redirect($location);
321| $this->redirect($e->getLocation());

/home/kali/Downloads/hackazon-master/classes/App/Controller/Install.php
>> php.symfony.security.audit.symfony-non-literal-redirect.symfony-non-literal-redirect
The 'redirect()' method does not check its destination in any way. If you redirect to a URL provided
by end-users, your application may be open to the unvalidated redirects security vulnerability.
Consider using literal values or an allowlist to validate URLs.
Details: https://sg.run/4ey5

23| $this->redirect($e->getLocation());
43| $this->redirect('/install/' . $step->getName());
48| $this->redirect('/install/' . $step->getNextStep()->getName());
54| $this->redirect('/install/' . $step->getName());

/home/kali/Downloads/hackazon-master/classes/App/Controller/Review.php
>> php.symfony.security.audit.symfony-non-literal-redirect.symfony-non-literal-redirect
The 'redirect()' method does not check its destination in any way. If you redirect to a URL provided
by end-users, your application may be open to the unvalidated redirects security vulnerability.
Consider using literal values or an allowlist to validate URLs.
Details: https://sg.run/4ey5

>>> php.lang.security.exec-use.exec-use
Executing non-constant commands. This can lead to command injection.
Details: https://sg.run/5Q1j

105| exec('type ' . $path, $content);
107| exec('cat ' . $path, $content);

/home/kali/Downloads/hackazon-master/classes/App/Controller/Checkout.php
>> php.symfony.security.audit.symfony-non-literal-redirect.symfony-non-literal-redirect
The 'redirect()' method does not check its destination in any way. If you redirect to a URL provided
by end-users, your application may be open to the unvalidated redirects security vulnerability.
Consider using literal values or an allowlist to validate URLs.
Details: https://sg.run/4ey5

279| $this->redirect($location);
321| $this->redirect($e->getLocation());

/home/kali/Downloads/hackazon-master/classes/App/Controller/Install.php
>> php.symfony.security.audit.symfony-non-literal-redirect.symfony-non-literal-redirect
The 'redirect()' method does not check its destination in any way. If you redirect to a URL provided
by end-users, your application may be open to the unvalidated redirects security vulnerability.
Consider using literal values or an allowlist to validate URLs.
Details: https://sg.run/4ey5

23| $this->redirect($e->getLocation());
43| $this->redirect('/install/' . $step->getName());
48| $this->redirect('/install/' . $step->getNextStep()->getName());
54| $this->redirect('/install/' . $step->getName());

/home/kali/Downloads/hackazon-master/classes/App/Controller/Review.php
>> php.symfony.security.audit.symfony-non-literal-redirect.symfony-non-literal-redirect
The 'redirect()' method does not check its destination in any way. If you redirect to a URL provided
by end-users, your application may be open to the unvalidated redirects security vulnerability.
Consider using literal values or an allowlist to validate URLs.
Details: https://sg.run/4ey5
```

Fig 9.5.2 – HACKAZON SCAN ANALYSIS

```
/home/kali/Downloads/hackazon-master/web/js/swfobject.js
>>> javascript:browser.security.insecure-document-method.insecure-document-method
User controlled data in methods like 'innerHTML', 'outerHTML' or 'document.write' is an anti-pattern
that can lead to XSS vulnerabilities
Details: https://sg.run/LwA9

386| ac.innerHTML = obj.innerHTML;
435| el.outerHTML = '<object classid="clsid:D27CDB6E-AE6D-11cf-96B8-444553540000"' + att + '>'
+ par + '</object>';

/home/kali/Downloads/hackazon-master/web/upload/download.php
>>> php.lang.security.injection.tainted-filename.tainted-filename
File name based on user input risks server-side request forgery.
Details: https://sg.run/Ayqp

30| if (!$file->loaded() || !file_exists($file->path) || !is_file($file->path)) {
30| if (!$file->loaded() || !file_exists($file->path) || !is_file($file->path)) {
37| $imgData = getimagesize($filePath);
39| header('Content-Length: ' . filesize($filePath));
40| readfile($filePath);

Scan Summary
Some files were skipped or only partially analyzed.
Partially scanned: 61 files only partially analyzed due to parsing or internal Semgrep errors
Scan skipped: 1 files larger than 1.0 MB, 4395 files matching .semgrepignore patterns
For a full list of skipped files, run semgrep with the --verbose flag.

Ran 712 rules on 1559 files: 170 findings.
Too many findings? Try Semgrep Pro for more powerful queries and less noise.
See https://sg.run/false-positives.

(kali@kali) ~
```

Fig 9.5.3 – HACKAZON SEMGREP SCAN FINDING AND SUCCESS